

Panel: NDN Application Development: Lessons Learned and Next Steps

Alex Afanasyev (FIU)

NDNComm 2021, NIST (Virtual)
October 28-29

Codebase Exists

- Beichuan highlighted the extent of the NDN codebase that was developed over the years. If categorizing:
 - Forwarder
 - Another forwarder
 - Another forwarder
 - Another forwarder?
 - Base library (aka low level NDN primitives)
 - Another library (aka low level NDN primitives)
 - Another library (aka low level NDN primitives)
 - Another library (aka low level NDN primitives)
 - Another library (aka low level NDN primitives)
 - Another library (aka low level NDN primitives)
 - Another library (aka low level NDN primitives)
 - A few higher level libraries (*sync)
 - A few network applications (*repo, *tools, *routing)
 - **A few application and application focused libs**
 - **A few early adopters trying to figure out how to use the codebase**
 - **A few early adopters making app focused codebase**
- I may be missing many
- We need these more
- 

My High-Level Take on Issues

- We have been stuck (probably still stuck) in the cycle of forwarder + core libraries
- Unclear (and in a few cases conflicting) targets from the early adoption for the above
 - do we focus on performance or modularity
 - do we prioritize “infrastructure” based or ad hoc
 - (can talk for myself) very limited exposure to actual users
- **Very limited developer manpower**
 - We don't have FTEs
 - Students come and go, many have specific focus on research project
 - Over the years, we had so many ideas to improve codebase, solve issues, improve usability, security, etc. and not happening cause limited manpower

(1) Major pain points

Just one of the points, but is quite common perception

(potential incorrect perception) NFD, cxx, and other core library don't accept external contribution

- Not true. We are open source. But we also trying (to the best of our ability) to ensure quality and scope of the changes:
 - does the change impact design decision + goals (some currently conflicting)
 - is the implementation complete
 - test coverage
- To fix these takes time and effort (we don't have FTEs for that), accepted code would be maintained by "us" (we actually still don't have FTEs for that)
 - Does the community want "us" to stop ensuring the quality?
- My best advise:
 - Submit the code
 - Explain the code and design decisions (either in the code or in our public weekly NDN calls)
 - Ensure quality of your code and push for it to be included
 - We still don't have FTEs for that

(2) Improve the developer's experience

- Need higher level APIs
 - But also need a design discussion of what actually need to be exposed
 - NDN use names, security based on names, transport based on names. Can they be completely hidden in “libraries” ?
 - This cannot be answered without actually making and running apps. If they can work perfectly, efficiently, and be secured without exposure, then answer yes.
- Need usability and automation
 - Others already highlighted, part of the problem is too low-level focus of majority of the libraries
 - Security management is a huge issue and in a dire need to be improved
 - Guidelines and suggested mechanics for bootstrapping
 - Automation of signing/verification with schemas
 - Automation of schemas themselves
 - Auto key management (e.g., publishing)
- Yet we still don't have FTE for that...

(3) Action Items

- Improve conversation between early adopters and <??> to push for higher level (new/old) APIs, usability, etc.
 - mailing list
 - weekly public NDN calls
 - slack
 - direct email
- Find more early adopters to improve things
 - Implement and share new things
- Discuss goals for NDN codebase by NDN team and how extremely limited resources of **volunteer** devs can be used
 - Pick a direction. At least NDN team effort now is way too fractured
 - Pick priority environment
 - ad hoc, native infrastructure, overlay: usability impl will differ wildly